code, explanation

How to run it

Clone the picorv32 repository into a folder. Add the files from the following folder, it has activation function code and also testbenches we have used for testing. The testbenches contain 120 instructions with 30 values(all 4 new instructions with every value) given as a testcase(checkpoint 1)
HackRush_7_Part2

In Vivado, add the picorv32.v, pcpi_ml_activations.v as the design sources and testbenches as required

Code Explanation

pcpi_ml_activations module takes in clk,rst, rs1,rs2 and instruction and pcpi_valid as inputs, it gives some control signals like pcpi_wait, pcpi_ready and pcpi_wr, the output data is in pcpi_rd

There are many localparam initialized to increase the readability of the actual logic
op_relu, op_sigm … are set when the instruction is of that particular type

The following code has a 3-state FSM, as written in the high-level explanation. ReLU and LReLU are not very math-intensive, so there is no 2nd state in the FSM; we complete them in 2 cycles(op_fast is set when the instruction is relu or l_relu)

For relu:
[-inf,0): 0
[0,inf): x

For leaky relu: same as relu but ⅛ * x when x<0

For sigmoid:
[0,1): 0.5 + |x|/4
[1,2): 0.75 + (|x|-1)/8
[2,4): 0.875 + (|x|-2)/16
[4,inf): 1.0

For tanh:

We use the same functions as sigmoid and use the relation between tanh and sigmoid

We also have Q16.16 fixed-point constants as localparams to increase readability, we have 0,½,¾,⅞,1,-1,2 and also max and min values(2^31 - 1 and -2^31)

We use sigmoid function for tanh so incase of overflow when we multiply by 2, we just use max and min values that we have as localparams

To check for threshold, we use this, which is better than using comparators

```
// Threshold checks for |x| >= 1,2,4 in Q16.16 using bit-slices
(LUT-friendly).
    wire ge_1 = |nl_abs[31:16];
    wire ge_2 = |nl_abs[31:17];
    wire ge_4 = |nl_abs[31:18];
```

In the FSM

ST_IDLE:
Waits for a valid custom activation request: pcpi_valid && instr_any.
If request is ReLU/LReLU (op_fast):
computes fast result immediately from current input
stores it in result_q
sets pcpi_wait = 1
moves directly to ST_DONE (skips compute stage)
If request is Sigmoid/Tanh:
latches operand into rs1_q
latches operation type into op_q
sets pcpi_wait = 1
moves to ST_COMPUTE

ST_COMPUTE:
Used only for Sigmoid/Tanh.
Keeps pcpi_wait = 1 (CPU stays stalled).
Runs non-linear datapath from registered inputs and writes result into result_q.
Moves to ST_DONE.

ST_DONE:
Completes handshake to CPU for one cycle:
pcpi_ready = 1
pcpi_wr = 1
pcpi_rd = result_q
Releases stall (pcpi_wait stays low by default).

Returns to ST_IDLE for next instruction.

So latency is:
ReLU/LReLU: 2-stage path (IDLE -> DONE)
Sigmoid/Tanh: 3-stage path (IDLE -> COMPUTE -> DONE)

```
// Integer-only ML activation coprocessor for PicoRV32 PCPI.
// Implements Q16.16 RELU, L_RELU, SIGM_APPROX, TANH_APPROX.

module pcpi_ml_activations (
    input             clk,
    input             resetn,
    input             pcpi_valid,
    input      [31:0] pcpi_insn,
    input      [31:0] pcpi_rs1,
    input      [31:0] pcpi_rs2,
    output reg        pcpi_wr,
    output reg [31:0] pcpi_rd,
    output reg        pcpi_wait,
    output reg        pcpi_ready
);
    localparam [6:0] OPCODE_CUSTOM0 = 7'b0001011;
    localparam [2:0] F3_ACT = 3'b000;

    localparam [6:0] F7_RELU = 7'd0;
    localparam [6:0] F7_LRELU = 7'd1;
    localparam [6:0] F7_SIGM = 7'd2;
    localparam [6:0] F7_TANH = 7'd3;
    // Internal operation encoding captured in IDLE and used in later FSM
states.
    localparam [1:0] OP_RELU = 2'd0;
    localparam [1:0] OP_LRELU = 2'd1;
    localparam [1:0] OP_SIGM = 2'd2;
    localparam [1:0] OP_TANH = 2'd3;

    // 3-cycle PCPI FSM:
    // IDLE    -> capture request and assert wait
    // COMPUTE -> evaluate datapath and register output
```

```
    // DONE    -> return ready/wr + result to the CPU
    localparam [1:0] ST_IDLE = 2'd0;
    localparam [1:0] ST_COMPUTE = 2'd1;
    localparam [1:0] ST_DONE = 2'd2;

    // Q16.16 fixed-point constants.
    localparam signed [31:0] Q16_ZERO = 32'sd0;
    localparam signed [31:0] Q16_HALF = 32'sd32768;     // 0.5
    localparam signed [31:0] Q16_THREE_QUARTER = 32'sd49152; // 0.75
    localparam signed [31:0] Q16_SEVEN_EIGHTH = 32'sd57344;  // 0.875
    localparam signed [31:0] Q16_ONE = 32'sd65536;      // 1.0
    localparam signed [31:0] Q16_NEG_ONE = -32'sd65536; // -1.0
    localparam signed [31:0] Q16_ONE_X2 = 32'sd131072;  // 2.0
    // Integer saturation bounds used for overflow-safe intermediate
operations.
    localparam signed [31:0] INT32_MIN = 32'sh8000_0000;
    localparam signed [31:0] INT32_MAX = 32'sh7fff_ffff;

    // Decode only our custom activation instruction family:
    // opcode=custom0 and funct3=000, funct7 selects operation.
    wire dec_custom = pcpi_insn[6:0] == OPCODE_CUSTOM0;
    wire dec_f3 = pcpi_insn[14:12] == F3_ACT;
    wire [6:0] dec_f7 = pcpi_insn[31:25];
    wire dec_act = dec_custom && dec_f3;
    wire op_relu = dec_act && (dec_f7 == F7_RELU);
    wire op_lrelu = dec_act && (dec_f7 == F7_LRELU);
    wire op_sigm = dec_act && (dec_f7 == F7_SIGM);
    wire op_tanh = dec_act && (dec_f7 == F7_TANH);
    wire instr_any = op_relu || op_lrelu || op_sigm || op_tanh;
    wire op_fast = op_relu || op_lrelu;

    reg [1:0] state;
    reg [1:0] op_q;
    reg signed [31:0] rs1_q;
    reg signed [31:0] result_q;
    wire signed [31:0] pcpi_rs1_s = pcpi_rs1;
    wire signed [31:0] relu_fast = pcpi_rs1[31] ? Q16_ZERO : pcpi_rs1_s;
    wire signed [31:0] lrelu_fast = pcpi_rs1[31] ? (pcpi_rs1_s >>> 3) :
pcpi_rs1_s;
```

```
    // Registered operand is used in COMPUTE to break long comb paths from
CPU inputs.
    wire signed [31:0] rs1_s = rs1_q;

    wire op_q_tanh = (op_q == OP_TANH);
    wire op_q_sigm = (op_q == OP_SIGM);
    // tanh(x) uses sigmoid(2x); saturate 2x to avoid wraparound on
overflow.
    wire x2_pos_ovf = ~rs1_s[31] & rs1_s[30];
    wire x2_neg_ovf = rs1_s[31] & ~rs1_s[30];
    wire signed [31:0] rs1_x2_sat =
        x2_pos_ovf ? INT32_MAX :
        x2_neg_ovf ? INT32_MIN :
        (rs1_s <<< 1);
    wire signed [31:0] nl_x = op_q_tanh ? rs1_x2_sat : rs1_s;
    // Saturating abs() handles INT32_MIN corner case (cannot be negated
in 32-bit signed).
    wire signed [31:0] nl_abs =
        nl_x[31] ? ((nl_x == INT32_MIN) ? INT32_MAX : -nl_x) : nl_x;

    // Threshold checks for |x| >= 1,2,4 in Q16.16 using bit-slices
(LUT-friendly).
    wire ge_1 = |nl_abs[31:16];
    wire ge_2 = |nl_abs[31:17];
    wire ge_4 = |nl_abs[31:18];

    // Shared multi-segment PWL sigmoid approximation:
    // [0,1): 0.5 + |x|/4
    // [1,2): 0.75 + (|x|-1)/8
    // [2,4): 0.875 + (|x|-2)/16
    // [4,inf): 1.0
    wire signed [31:0] sigm_abs_seg =
        ge_4 ? Q16_ONE :
        ge_2 ? (Q16_SEVEN_EIGHTH + ((nl_abs - Q16_ONE_X2) >>> 4)) :
        ge_1 ? (Q16_THREE_QUARTER + ((nl_abs - Q16_ONE) >>> 3)) :
               (Q16_HALF + (nl_abs >>> 2));

    // Mirror around 0 for negative input, then derive tanh from sigmoid
identity.
```

```
    wire signed [31:0] sigm_val = nl_x[31] ? (Q16_ONE - sigm_abs_seg) :
sigm_abs_seg;
    wire signed [31:0] tanh_val = (sigm_val <<< 1) - Q16_ONE;

    wire signed [31:0] relu_val = rs1_s[31] ? Q16_ZERO : rs1_s;
    wire signed [31:0] lrelu_val = rs1_s[31] ? (rs1_s >>> 3) : rs1_s;
    // Final operation mux. One shared non-linear datapath is reused by
SIGM/TANH.
    wire signed [31:0] result =
        (op_q == OP_RELU) ? relu_val :
        (op_q == OP_LRELU) ? lrelu_val :
        op_q_sigm ? sigm_val :
        op_q_tanh ? tanh_val :
        32'sd0;

    always @(posedge clk) begin
        pcpi_wr <= 1'b0;
        pcpi_ready <= 1'b0;
        pcpi_wait <= 1'b0;
        pcpi_rd <= 32'b0;

        if (!resetn) begin
            // Reset to known IDLE outputs/state.
            state <= ST_IDLE;
            op_q <= OP_RELU;
            rs1_q <= 32'sd0;
            result_q <= 32'sd0;
        end else begin
            case (state)
                ST_IDLE: begin
                    if (pcpi_valid && instr_any) begin
                        // Accept request and hold CPU with wait while we
process.
                        pcpi_wait <= 1'b1;
                        if (op_fast) begin
                            // Fast path: RELU/LRELU result is directly
available, skip COMPUTE.
                            result_q <= op_relu ? relu_fast : lrelu_fast;
                            state <= ST_DONE;
                        end else begin
```

```
                            rs1_q <= pcpi_rs1;
                            if (op_sigm)
                                op_q <= OP_SIGM;
                            else
                                op_q <= OP_TANH;
                            state <= ST_COMPUTE;
                        end
                    end
                end

                ST_COMPUTE: begin
                    // Register datapath result to isolate timing from
response phase.
                    pcpi_wait <= 1'b1;
                    result_q <= result;
                    state <= ST_DONE;
                end

                ST_DONE: begin
                    // One-cycle completion pulse back to PicoRV32.
                    pcpi_ready <= 1'b1;
                    pcpi_wr <= 1'b1;
                    pcpi_rd <= result_q;
                    state <= ST_IDLE;
                end

                default: begin
                    state <= ST_IDLE;
                end
            endcase
        end
    end

    // Silence lint for currently unused rs2 in this instruction family.
    wire _unused_ok = &{1'b0, pcpi_rs2, 1'b0};

endmodule
```

explanation, LUTs, power report

Eshwar, Vignesh

Activation functions logic

Note - the explanation written below is refined using AI

1. Basic Logic & Architecture

The `pcpi_ml_activations` module acts as a custom hardware accelerator attached to the PicoRV32 core via the Pico Co-Processor Interface (PCPI).

To handle fractional values, the module strictly utilizes the Q16.16 fixed-point number format.

To safely integrate with the main CPU without destroying its maximum clock frequency, the coprocessor operates using an FSM:

ST_IDLE: The module waits for a valid custom instruction. Once received, it captures the input data into internal registers and asserts `pcpi_wait` to stall the main CPU.

ST_COMPUTE: The module allows the combinatorial math logic to evaluate the activation function and registers the final answer.

ST_DONE: The module drops the wait signal, asserts `pcpi_ready`, and writes the computed data back to the CPU's destination register.

2. Mathematical Approximations

We use Piecewise functions for linear approximations.

The continuous non-linear curves are broken down into discrete linear segments. To avoid using power-hungry and area-heavy DSP multipliers, the slopes of these linear segments are strictly constrained to powers of two (e.g., 1/2, 1/4, 1/8). This allows the hardware to compute the slopes using simple arithmetic right-shifts (`>>>`).

For example, the Sigmoid function is approximated by taking the absolute value of the input (|x|) and passing it through four distinct thresholds:
|x| >= 4: Output is clamped to 1.0.
|x| >= 2: Evaluates as 0.875 + (|x|-2)/16.
|x| >= 1: Evaluates as 0.75 + (|x|-1)/8.
|x| < 1: Evaluates as 0.5 + |x|/4.

The result is then mirrored around 0.5 if the original input was negative.

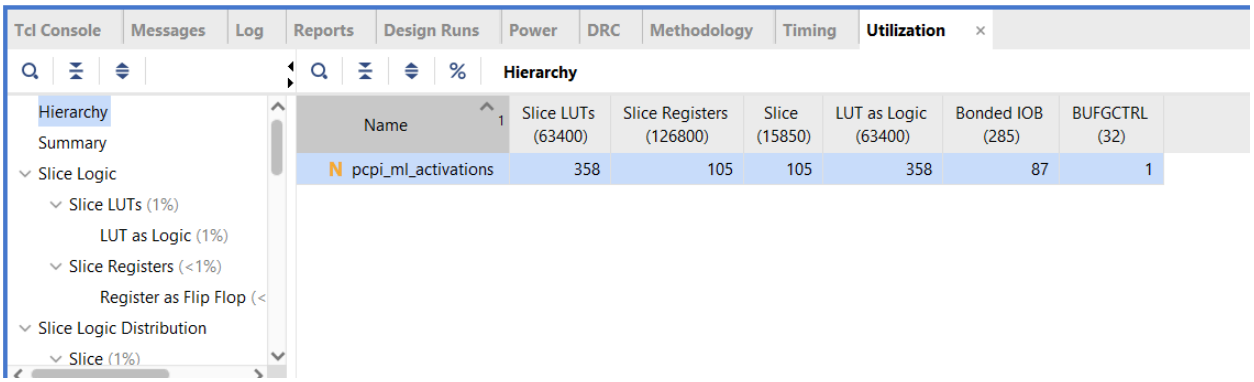
3. Key Hardware Optimizations

To maximize the evaluation score across Area, Latency, and Power, three major architectural optimizations were implemented:

- **Mathematical Resource Sharing (Area Optimization):** Instead of instantiating two massive, separate logic blocks for Sigmoid and Tanh, the design exploits the mathematical identity:
  tanh(x) = 2*sigmoid(2x) - 1
  When a Tanh instruction is decoded, the hardware simply shifts the input left by 1 (multiplying by 2), routes it through the shared Sigmoid datapath, and then shifts the output left by 1 and subtracts 1.0. This effectively halves the LUT utilization for non-linear operations.
- **Dynamic Latency Fast-Path (Timing Optimization):**
  While complex non-linear functions require the full 3-cycle FSM to safely resolve, linear functions like ReLU and Leaky ReLU are mathematically shallow. The FSM includes a dynamic bypass (`op_fast`) that routes these linear instructions directly from `ST_IDLE` to `ST_DONE`. This optimization reduces the latency of ReLU operations from 3 cycles to 2, improving the overall Instructions Per Clock (IPC) during neural network execution.
- **Bitwise Threshold Reduction (LUT Optimization):**
  In the PWL approximation, the hardware must determine which segment the input falls into (e.g., checking if $x \ge 1.0$, 2.0, or 4.0). Using standard 32-bit hardware comparators (`>=`) consumes a large number of logic gates. Instead, the design uses bitwise OR reductions on specific slices of the Q16.16 integer bits (e.g., `|nl_abs[31:16]`). If any bit in the integer portion is a 1, the value is guaranteed to be >1.0. This trick drastically reduces the gate count while maintaining perfect logical accuracy.

The code was synthesized and implemented successfully

Utilization Design Information

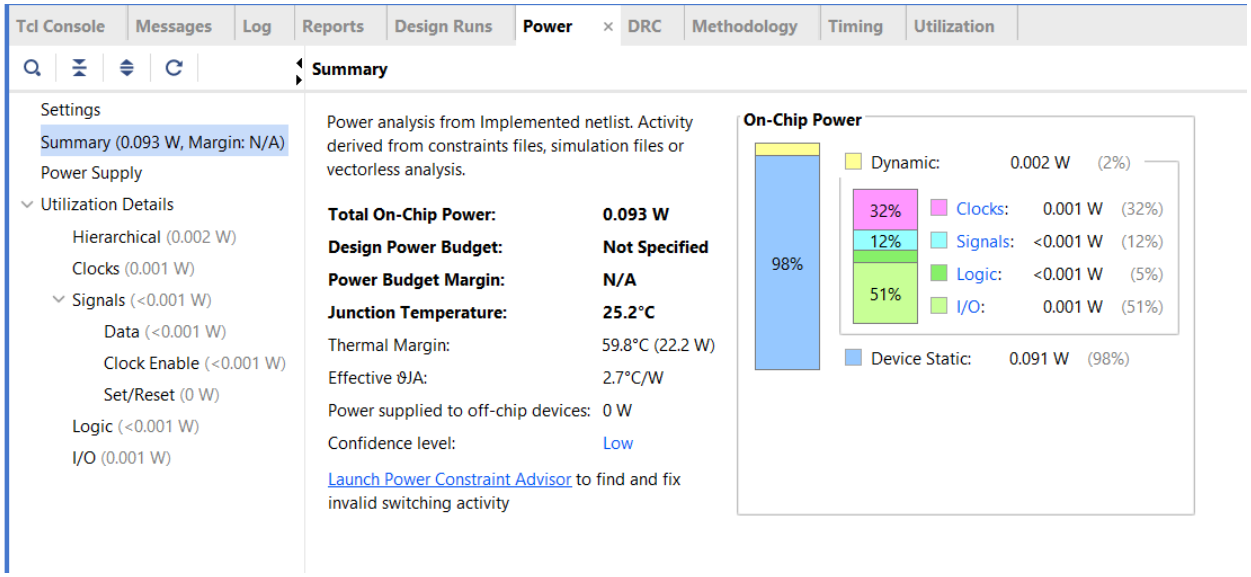


1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs	358	0	63400	0.56
LUT as Logic	358	0	63400	0.56
LUT as Memory	0	0	19000	0.00
Slice Registers	105	0	126800	0.08
Register as Flip Flop	105	0	126800	0.08
Register as Latch	0	0	126800	0.00
F7 Muxes	0	0	31700	0.00
F8 Muxes	0	0	15850	0.00

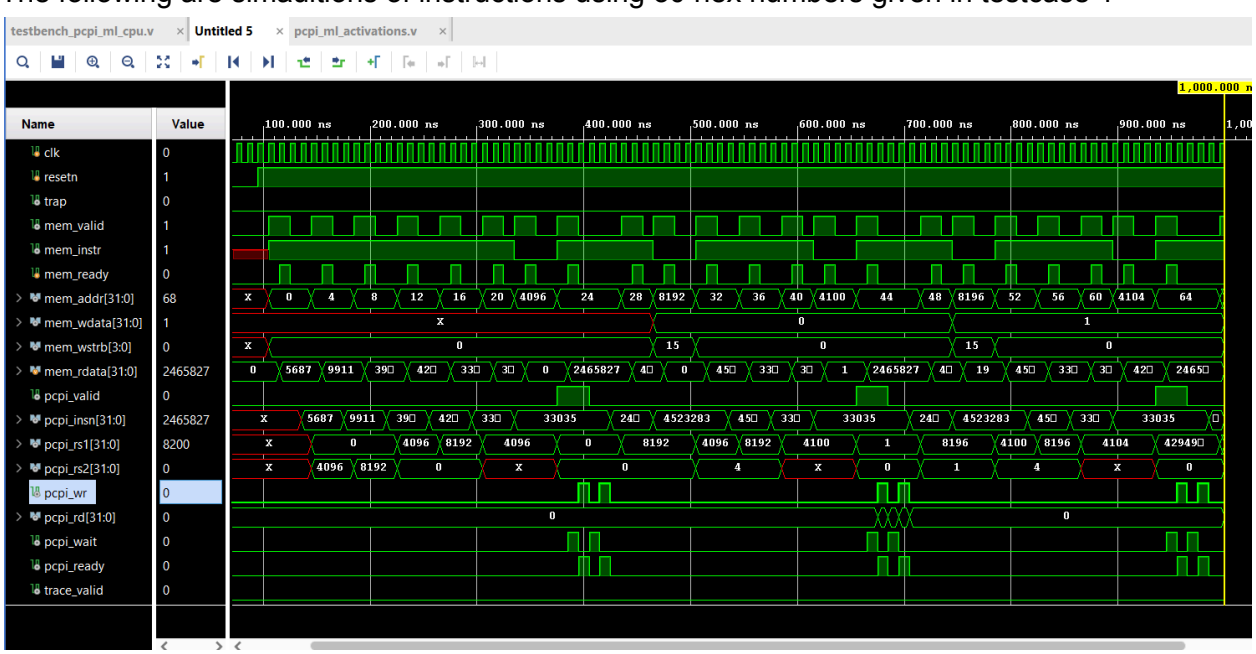
Eshwar, Vignesh

Power Report



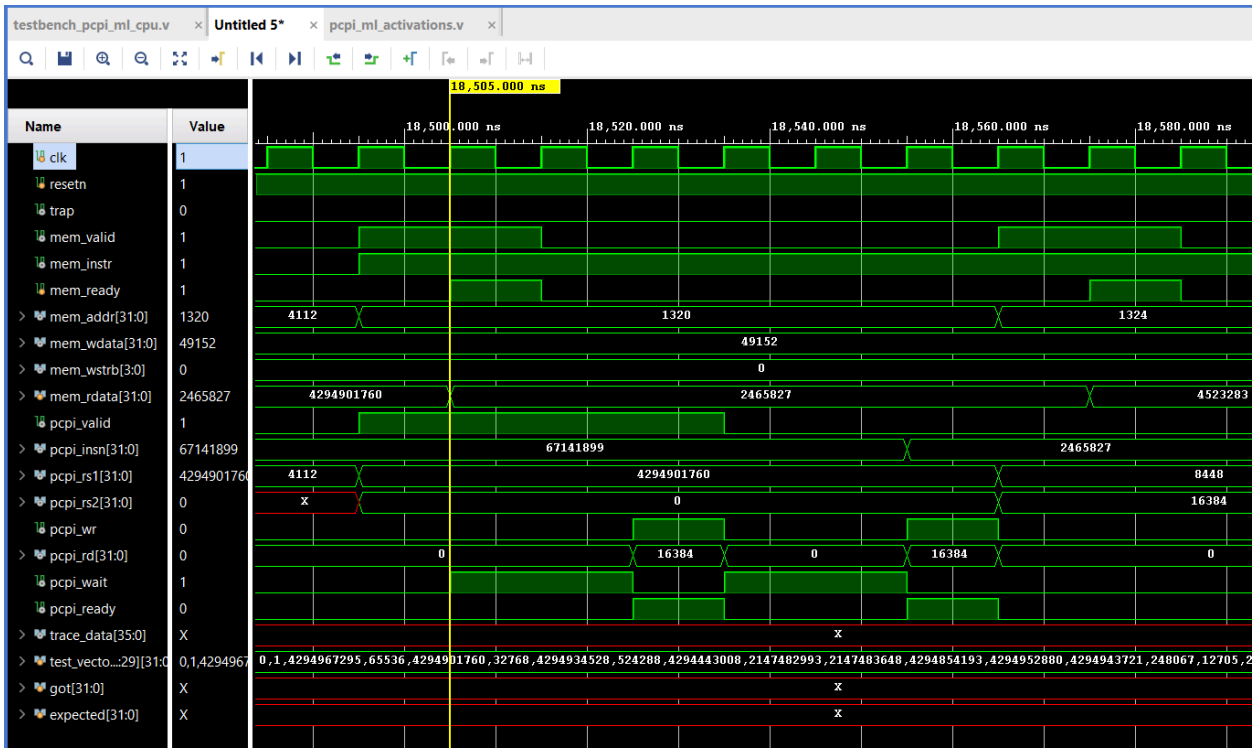
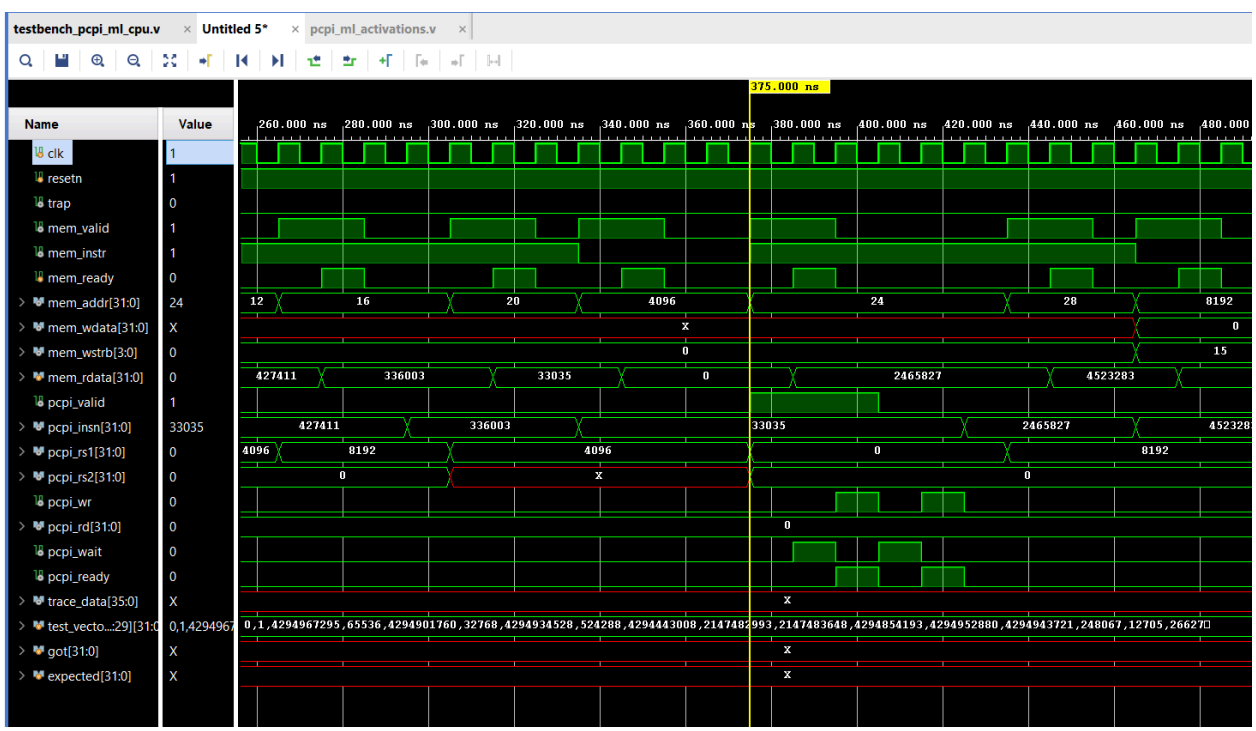
Simulation results

The following are simaultions of instructions using 30 hex numbers given in testcase 1



Eshwar, Vignesh

This shows the 3 cycle exec of sigmoid instruction, the pcpi_wait is 1 for 2 cycles and pcpi_ready is 1 for 3rd cycle, the output value 16384 which is 0.25



This shows the 2 cycle execution of relu(pcpi_rd stays 0 as input was 0) the pcpi_wait is 1 only for 1 cycle then pcpi_ready is 1